

Titre du cours : Mise en place d'un
écosystème d'IA
Code officiel : 420-A57-BB
Professeur : Dr Komi SODOKE

MODULE 01

Cerner les besoins matériels dans le contexte de l'IA

Spécialisation technique en Intelligence Artificielle

Chaîne CI/CD

Composantes matérielles IA

PLAN DU COURS

Objectif terminal : Cerner les besoins matériels dans le contexte de l'IA

I

Introduction — Pourquoi le matériel est crucial pour l'IA

II

Composantes d'une chaîne CI/CD pour l'IA
(Code source · Serveurs CI · Données · Besoins matériels)

III

Composantes matérielles nécessaires à l'IA
(CPU · GPU · TPU · Optimisations · Coûts · Durée de vie)

IV

Quiz de compréhension

V

Conclusion & Synthèse

Pourquoi l'IA est exigeante en ressources ?

Contrairement à un logiciel classique (tableur, site web), l'IA — et surtout le Deep Learning — effectue des milliards d'opérations mathématiques simultanées sur des matrices de données. Ces calculs ne peuvent pas se faire efficacement sur un simple ordinateur de bureau.

Calcul tensoriel massif

L'IA multiplie des matrices de plusieurs millions de dimensions. Ex : un modèle GPT-3 contient 175 milliards de paramètres à calculer à chaque passe d'entraînement.

L'IA multiplie des matrices de plusieurs millions de dimensions. Ex : un modèle GPT-3 contient 175 milliards de paramètres à calculer à chaque passe d'entraînement.

Données volumineuses

Un seul dataset d'images pour la vision par ordinateur peut dépasser plusieurs téraoctets. Ces données doivent être transférées rapidement vers le processeur sans goulot d'étranglement.

Un seul dataset d'images pour la vision par ordinateur peut dépasser plusieurs téraoctets. Ces données doivent être transférées rapidement vers le processeur sans goulot d'étranglement.

Scalabilité nécessaire

Un modèle simple s'entraîne sur un PC. Un LLM (Grand Modèle de Langage) nécessite des milliers de GPU en parallèle. L'infrastructure doit pouvoir grandir.

Un modèle simple s'entraîne sur un PC. Un LLM (Grand Modèle de Langage) nécessite des milliers de GPU en parallèle. L'infrastructure doit pouvoir grandir.

I — Les trois piliers d'une infrastructure IA

Section I

Tout projet d'IA performant repose sur trois piliers indissociables. Négliger l'un d'eux compromet l'ensemble du projet.

01 Puissance de calcul

- Le calcul est le cœur de l'IA. Sans GPU puissant, l'entraînement d'un réseau de neurones qui prend 2h peut en prendre 2 semaines sur CPU.
- Les GPU modernes (ex: NVIDIA A100) effectuent jusqu'à 312 TFLOPS (téraflops), soit 312 000 milliards d'opérations par seconde.
- Les TPU de Google peuvent dépasser 420 TFLOPS spécifiquement pour les opérations tensorielles.
- Règle : plus le modèle est grand, plus la puissance de calcul nécessaire est élevée.

02 Gestion des données

- L'entraînement IA consomme des quantités massives de données (images, textes, sons). Ces données doivent être accessibles rapidement.
- Un SSD NVMe peut lire à 7 Go/s, contre 550 Mo/s pour un SSD SATA et 150 Mo/s pour un HDD. Le choix du stockage change tout.
- Le principe MLOps : 'même code + données différentes = résultats différents'. Les données doivent être versionnées.
- Les solutions comme DVC, MLflow permettent de tracer chaque expérience et de reproduire les résultats.

03 Scalabilité & Flexibilité

- Un projet IA démarre souvent petit (1 GPU) et doit pouvoir évoluer vers des clusters (100+ GPU) sans refaire l'architecture.
- La virtualisation (Docker, Kubernetes) permet d'isoler les environnements et de déployer partout de façon identique.
- Le cloud (AWS, GCP, Azure) permet de louer de la puissance à la demande : idéal pour des pics de calcul ponctuels.
- L'infrastructure doit supporter l'automatisation : déclenchement automatique de l'entraînement dès qu'une donnée change.

I — Objectifs du module & compétences visées

Section I

À l'issue de ce module, vous serez capable de :

Comprendre

Expliquer pourquoi les projets d'IA nécessitent une infrastructure spécifique, différente des applications web ou mobiles classiques.

Identifier

Nommer et décrire le rôle de chaque composante d'une chaîne CI/CD adaptée à l'IA (Git, DVC, Jenkins, MLflow, Docker, Kubernetes).

Analyser

Comparer les performances de CPU, GPU et TPU pour différents scénarios : entraînement intensif, inférence légère, prétraitement de données.

Optimiser

Proposer une configuration matérielle équilibrée en tenant compte des goulots d'étranglement (PCIe, RAM, stockage, ratio CPU/GPU).

Évaluer

Estimer les coûts matériels, les coûts d'exploitation (électricité, refroidissement) et l'espérance de vie des composants pour guider les décisions d'achat.

Choisir

Choisir entre infrastructure on-premise et cloud computing selon les besoins réels d'un projet IA concret.

SECTION II

Composantes d'une chaîne CI/CD pour l'IA

Intégration et déploiement continus · MLOps

Git / GitHub

Jenkins / GitLab CI

DVC / MLflow

Docker / K8s

II — Qu'est-ce que la CI/CD ? Définition fondamentale

CI/CD

La CI/CD, c'est quoi ?

CI/CD signifie Continuous Integration / Continuous Deployment. C'est un ensemble de pratiques et d'outils qui automatisent le cycle de vie d'un logiciel — de l'écriture du code jusqu'à sa mise en production. En IA, on parle souvent de MLOps (Machine Learning Operations) pour désigner l'adaptation de ces pratiques aux modèles de Machine Learning.

CI

Intégration
Continue

Chaque développeur intègre son code fréquemment (plusieurs fois par jour). À chaque 'commit', des tests automatiques sont déclenchés pour détecter immédiatement les erreurs.

Exemple : un data scientist modifie l'architecture d'un réseau de neurones → Jenkins lance automatiquement les tests unitaires et vérifie que le code compile correctement.

CD

Déploiement
Continu

Une fois les tests passés, le modèle ou l'application est automatiquement déployé en production (ou en pré-production). Cela élimine les déploiements manuels risqués.

Exemple : un nouveau modèle de classification d'images passe tous les tests de performance → il est automatiquement mis à jour sur l'API de production.

II — CI/CD classique vs CI/CD pour l'IA (MLOps)

CI/CD

En développement logiciel classique, la CI/CD gère le code source. En IA, elle doit gérer trois éléments supplémentaires : les données, les modèles et les expériences. C'est cette complexité qui a donné naissance au MLOps.

Aspect	CI/CD Classique	CI/CD pour l'IA (MLOps)
Ce qui est versionné	Le code source uniquement	Code + Données + Modèles + Hyperparamètres
Tests automatiques	Tests unitaires, tests d'intégration	Tests de code + Tests de performance du modèle (accuracy, F1-score...)
Artefacts produits	Un binaire, une application web	Un modèle entraîné (.pkl, .pt, .h5) + ses métriques
Reproductibilité	Même code = même résultat	Même code + mêmes données + mêmes seeds = même modèle
Infrastructure requise	Serveurs CPU standards	GPU/TPU + stockage haute vitesse + réseau haut débit
Outils spécifiques	Git, Jenkins, Docker	Git + DVC + MLflow + Kubeflow + Seldon

Retenir : En IA, l'automatisation doit couvrir toute la chaîne : ingestion des données → entraînement → évaluation → déploiement → monitoring.

II.1 — Gestionnaires de code source : Git & plateformes

CI/CD

Un gestionnaire de code source (VCS – Version Control System) est un logiciel qui enregistre l'historique complet de toutes les modifications apportées à un projet. Il permet à plusieurs personnes de collaborer sans écraser le travail des autres.

Git — L'outil de base incontournable

- 1 Créé en 2005 par Linus Torvalds (créateur de Linux), Git est aujourd'hui utilisé dans 95% des projets de développement professionnel.
- 2 Principe : chaque 'commit' est un instantané (snapshot) du projet à un moment précis. On peut revenir à n'importe quel état antérieur.
- 3 Les 'branches' permettent de développer une nouvelle fonctionnalité en parallèle sans impacter le code principal (branche 'main').
- 4 En IA : une branche par expérience ! Chaque tentative d'architecture de réseau de neurones peut être isolée dans sa propre branche.

Plateformes cloud : GitHub, GitLab, Bitbucket

GitHub

La plus grande plateforme mondiale. Intégration CI/CD via 'GitHub Actions'. Idéal pour les projets open-source et les équipes distribuées.

GitLab

Solution complète avec CI/CD intégrée nativement. Peut s'héberger soi-même (self-hosted). Très utilisé en entreprise pour les pipelines MLOps.

Bitbucket

Appartient à Atlassian (Jira). Idéal si l'équipe utilise déjà Jira pour la gestion de projet. Intégration native avec les outils Atlassian.

II.1 — Les commandes Git essentielles pour l'IA

CI/CD

Voici les commandes Git que tout data scientist utilise au quotidien. Comprendre ce que chaque commande fait réellement est essentiel pour travailler en équipe sans perdre de données.

`git init`

Initialise un nouveau dépôt Git dans le dossier courant. Crée un dossier caché `.git` qui stocke tout l'historique.

Au démarrage de tout nouveau projet IA.

`git add .`

Prépare tous les fichiers modifiés pour le prochain commit. C'est la 'zone de transit' (staging area).

Avant chaque commit pour sélectionner ce qui sera sauvegardé.

`git push origin main`

Envoie vos commits locaux vers le serveur distant (GitHub/GitLab) pour les partager avec l'équipe.

Pour partager votre travail et déclencher le pipeline CI/CD.

`git clone <url>`

Télécharge un projet existant depuis une plateforme (GitHub, GitLab) vers votre machine locale, avec tout son historique.

Pour rejoindre un projet existant ou utiliser un modèle pré-entraîné.

`git commit -m 'msg'`

Crée un instantané permanent du projet avec un message descriptif. Ce snapshot est immuable et traçable.

Après chaque modification significative : 'feat: ajout couche Dropout au CNN'.

`git checkout -b exp/lstm-v2`

Crée une nouvelle branche et bascule dessus. Permet d'expérimenter sans toucher au code principal.

-Pour tester une nouvelle architecture LSTM ou un nouveau prétraitement.

II.2 — Gestionnaires de données : le data versioning

CI/CD

⚠ **Problème fondamental : Même code + données différentes = résultats différents ! En IA, les données font partie du code.**

Pourquoi versionner les données ?

Imaginez que vous entraînez un modèle de détection de tumeurs médicales. Trois mois plus tard, un collègue modifie 50 images du dataset pour améliorer les annotations. Vous relancez l'entraînement avec le même code — et le modèle donne des résultats complètement différents. Sans versionnage des données, impossible de savoir quelle version du dataset a produit quel modèle.

DVC

Data Version Control

Le Git des données. DVC crée des fichiers '.dvc' (légers, versionnables dans Git) qui pointent vers les vraies données stockées sur S3, GCS, Azure ou un NAS. Commandes similaires à Git : `dvc add`, `dvc push`, `dvc pull`.

```
dvc add data/images/  
git add data/images.dvc  
git commit -m 'dataset v2.1'
```

MLflow

Tracking des expériences

Enregistre automatiquement chaque run d'entraînement : hyperparamètres utilisés, métriques obtenues (accuracy, loss), modèle produit. Une interface web permet de comparer visuellement toutes les expériences.

```
mlflow.log_param('lr', 0.001)  
mlflow.log_metric('acc', 0.94)  
mlflow.sklearn.log_model(model,  
    'clf')
```

W&B

Weights & Biases

Alternative à MLflow, orientée visualisation. Génère automatiquement des courbes d'apprentissage, des matrices de confusion, des visualisations de données. Très populaire dans la recherche en Deep Learning.

```
wandb.init(project='mon-projet')  
wandb.log({'loss': 0.35, 'acc':  
    0.92})  
wandb.finish()
```

II.2 — Stockage des données : Data Lake vs Data Warehouse

CI/CD

En IA, les données viennent de sources très diverses (images, textes, capteurs, bases de données). Il existe deux grandes approches pour les stocker et les organiser avant l'entraînement.

Data Lake

Un Data Lake est un stockage brut, non structuré, qui conserve les données dans leur format original (images JPEG, fichiers CSV, logs JSON, vidéos MP4...).

Avantages :

- Stocke tout, dans tous les formats
- Très flexible pour l'exploration
- Coût réduit (stockage objet)
- Idéal pour le Machine Learning

⚠ Limites :

- Peut devenir un 'Data Swamp' si mal géré
- Données difficiles à trouver sans catalogue
- Moins adapté aux requêtes SQL classiques

Exemples : AWS S3, Azure Data Lake, Google Cloud Storage, MinIO (on-premise)

Data Warehouse

Un Data Warehouse est un stockage structuré et optimisé pour les requêtes analytiques. Les données y sont transformées, nettoyées et organisées en tables.

Avantages :

- Données propres et structurées
- Requêtes SQL très rapides
- Idéal pour les tableaux de bord
- Historique et cohérence garantis

⚠ Limites :

- Transformation coûteuse (ETL)
- Moins flexible pour le ML non structuré
- Coût plus élevé

Exemples : Google BigQuery, Amazon Redshift, Snowflake, Microsoft Synapse

II.3 — Serveurs CI : automatiser l'entraînement et les tests

CI/CD

Un serveur CI (Intégration Continue) surveille le dépôt de code et déclenche automatiquement une série d'actions dès qu'un développeur pousse une modification. En IA, ces actions incluent non seulement les tests du code, mais aussi l'entraînement et l'évaluation des modèles.

Le flux automatisé d'un pipeline CI/CD pour l'IA :



Jenkins

✓ **Gratuit, très personnalisable**

Le plus ancien et le plus flexible. Open-source, entièrement configurable via des 'Jenkinsfile'. Nécessite une infrastructure propre (on-premise). Grande communauté, milliers de plugins disponibles.

GitLab CI/CD

✓ **Tout-en-un (code + CI/CD)**

Intégré directement dans GitLab. Le pipeline se définit dans un fichier '.gitlab-ci.yml' à la racine du projet. Supporte les GPU runners. Interface visuelle très claire.

GitHub Actions

✓ **Marketplace riche, intégration GitHub**

Workflows définis en YAML directement dans le dépôt GitHub. Marketplace avec des milliers d'actions prêtes à l'emploi. Exécution sur les serveurs GitHub ou self-hosted.

Kubeflow Pipelines

✓ **Spécialisé Machine Learning**

Solution spécialisée pour les pipelines MLOps sur Kubernetes. Chaque étape (prétraitement, entraînement, évaluation) est un conteneur Docker séparé. Très scalable.

II.4 — Besoins matériels d'une infrastructure CI/CD pour l'IA

CI/CD

Une chaîne CI/CD IA ne peut pas tourner sur un simple ordinateur portable. Elle nécessite une infrastructure dédiée composée de plusieurs types de serveurs aux rôles bien distincts.

Serveur Master CI

4–8 cœurs CPU

16–32 GB RAM

500 GB SSD

Orchestre et surveille l'ensemble des pipelines. Reçoit les webhooks de GitHub/GitLab, distribue le travail aux agents, affiche le tableau de bord CI/CD.

💡 *N'exécute pas lui-même les jobs — délègue aux agents.*

Agent / Runner CPU

8–16 cœurs CPU

32–64 GB RAM

1–2 TB SSD

Exécute les tests unitaires, les builds Docker, le prétraitement des données. Traite les tâches qui ne nécessitent pas de GPU : vérification du code, génération de rapports.

💡 *Peut tourner dans un conteneur Docker ou sur une VM dédiée.*

Agent / Runner GPU

16–32 cœurs CPU

64–128 GB RAM

4 TB NVMe

Exécute les entraînements de modèles et les tests de performance. Se déclenche uniquement quand les données ou le code du modèle changent (économie de ressources).

💡 *Peut être sur-demande dans le cloud pour optimiser les coûts.*

Serveur Artefacts

4 cœurs CPU

16 GB RAM

10+ TB RAID

Stocke tous les modèles entraînés, les datasets versionnés, les images Docker, les logs d'expériences. C'est la mémoire long terme de la chaîne CI/CD.

💡 *Exemples : Nexus, JFrog Artifactory, MinIO, registre Docker privé.*

II.4 — Docker & Kubernetes : la virtualisation des environnements

CI/CD

L'un des problèmes les plus fréquents en IA : 'ça marche sur mon PC mais pas sur le serveur'. Docker et Kubernetes sont les solutions à ce problème.



Docker

Docker crée des 'conteneurs' : des environnements d'exécution isolés qui embarquent le code, les bibliothèques Python, les versions de CUDA et toutes les dépendances nécessaires.

- Un conteneur Docker est identique partout : sur votre laptop, sur le serveur CI, en production.
- Le 'Dockerfile' décrit précisément l'environnement : python 3.10, pytorch 2.0, cuda 12.1...
- Les images Docker sont versionnables et partageables via Docker Hub ou un registre privé.
- En IA : chaque expérience peut avoir son propre conteneur avec ses propres versions de bibliothèques.

```
FROM pytorch/pytorch:2.0.1-cuda11.7
RUN pip install transformers==4.30
COPY train.py .
CMD ['python', 'train.py']
```



Kubernetes (K8s)

Kubernetes orchestre des ensembles de conteneurs Docker sur un cluster de machines. Il gère automatiquement la répartition de charge, la scalabilité et la tolérance aux pannes.

- Déploie automatiquement plus de conteneurs si la charge augmente (scalabilité horizontale).
- Relance automatiquement un conteneur s'il tombe en panne (auto-healing).
- Gère la distribution des jobs d'entraînement sur plusieurs nœuds GPU.
- Kubeflow est la version spécialisée de K8s pour les pipelines Machine Learning.

```
apiVersion: batch/v1
kind: Job
metadata:
  name: training-job
spec: ...
```

SECTION III

Composantes matérielles nécessaires à l'IA

CPU · GPU · TPU · Optimisations · Coûts · Durée de vie

III.1 — Le CPU : cerveau polyvalent de l'ordinateur

CPU — Central Processing Unit (Unité Centrale de Traitement)

Le CPU est le processeur généraliste de l'ordinateur. Il est conçu pour exécuter des tâches complexes et variées de façon séquentielle ou avec peu de parallélisme. Un CPU moderne possède entre 4 et 128 cœurs, chaque cœur pouvant exécuter des instructions complexes très rapidement.

Architecture du CPU :

1	Peu de cœurs puissants :	4 à 128 cœurs, chacun optimisé pour les calculs séquentiels complexes. Chaque cœur a une grande mémoire cache (L1/L2/L3) pour accéder rapidement aux données.
2	Cache hiérarchique :	L1 (ultra-rapide, petite) → L2 → L3 (plus grande, partagée). Le CPU passe 60% de son temps à chercher des données dans ces caches.
3	Branch prediction :	Prédit les prochaines instructions pour ne pas perdre de cycles. Essentiel pour le code logiciel classique avec beaucoup de conditions (if/else).
4	Mémoire adressable :	Accède à toute la RAM système (jusqu'à 2+ TB). Peut traiter des datasets qui ne tiennent pas en VRAM GPU.

 En IA : le CPU gère le prétraitement (nettoyage, normalisation, augmentation) des données AVANT de les envoyer au GPU. Un CPU trop lent crée un goulot d'étranglement qui laisse le GPU en attente.

III.1 — Le GPU : moteur du Deep Learning

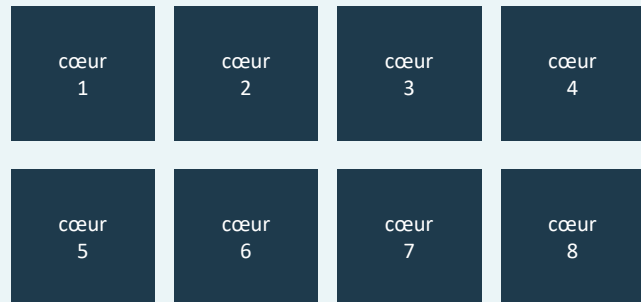
GPU — Graphics Processing Unit (Unité de Traitement Graphique)

Initialement conçu pour afficher des graphismes 3D (jeux vidéo), le GPU s'est révélé parfait pour l'IA car les deux tâches partagent la même opération fondamentale : la multiplication de matrices en parallèle. Un GPU peut effectuer des milliers d'opérations simultanément là où un CPU en fait quelques dizaines.

Pourquoi le GPU est 100x plus rapide que le CPU pour l'IA :

CPU (8 cœurs)

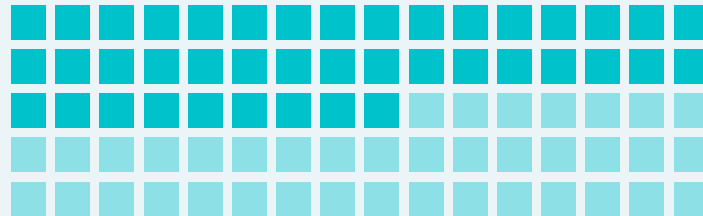
Calcul séquentiel



8 cœurs complexes

GPU (10 000 cœurs)

Calcul parallèle massif



*10 000+ cœurs simples
(représentation partielle)*

💡 *Analogie : le CPU est comme 8 brillants génies qui travaillent en séquence. Le GPU est comme 10 000 travailleurs simples qui font tous la même tâche en même temps. Pour une matrice de millions d'opérations identiques, les 10 000 gagnent à chaque fois.*

III.1 — GPU en IA : VRAM, architectures et modèles clés

La VRAM (Video RAM) est la mémoire embarquée dans le GPU. C'est le facteur limitant le plus critique : si votre modèle ne tient pas en VRAM, il ne peut tout simplement pas s'entraîner. Chaque couche du réseau, chaque batch de données, doit tenir dans cette mémoire.

Règle pratique VRAM :

VRAM nécessaire $\approx$ taille du modèle $\times$ 4 octets (float32) + données du batch + gradients ($\times 2$ en entraînement)

Modèle GPU	VRAM	TFLOPS (FP32)	Prix approx.	Usage recommandé
NVIDIA RTX 3060	12 GB	12.7 TFLOPS	~350 \$	Apprentissage, petits modèles
NVIDIA RTX 4090	24 GB	82.6 TFLOPS	~1 800 \$	Recherche, fine-tuning LLM
NVIDIA A100 (40GB)	40 GB	77.6 TFLOPS	~10 000 \$	Production data center
NVIDIA A100 (80GB)	80 GB	77.6 TFLOPS	~15 000 \$	Grands modèles (LLM, diffusion)
NVIDIA H100 SXM	80 GB	66.9 TFLOPS	~30 000 \$	Entraînement LLM state-of-the-art
AMD RX 7900 XTX	24 GB	61.4 TFLOPS	~1 000 \$	Alternative budget grand public

 Attention : plusieurs GPU peuvent être connectés (SLI/NVLink) pour cumuler la VRAM. $2 \times \text{A100 } 80\text{GB} = 160\text{GB}$ de VRAM effective, permettant d'entraîner des modèles comme LLaMA-70B.

III.1 — Le TPU : la puce spécialisée pour l'IA

TPU — Tensor Processing Unit (Unité de Traitement Tensoriel)

Le TPU est un circuit intégré spécifique (ASIC – Application Specific Integrated Circuit) conçu par Google uniquement pour accélérer les calculs d'IA, notamment les opérations tensorielles (multiplications de matrices). Contrairement au GPU qui reste polyvalent, le TPU ne fait qu'une chose — mais il la fait de façon extrêmement optimisée.

Comment fonctionne un TPU ?

Un TPU contient des milliers de 'Multiply-Accumulate Units' (MAC) organisées en systolic array (réseau systolique). Les données circulent dans ce réseau comme une chaîne de montage, permettant des multiplications de matrices massivement parallèles sans avoir à les charger/décharger en mémoire.

Avantages vs GPU

- ✓ Vitesse : 3 à 5× plus rapide que GPU équivalent sur TensorFlow/JAX.
- ✓ Efficacité énergétique : meilleur rapport FLOPS/Watt.
- ✓ Mémoire bande passante très élevée : idéal pour les LLM.
- ⚠ Limites : fonctionne uniquement avec TensorFlow et JAX (pas PyTorch natif).

TPU v4 de Google : chiffres clés

Performances : 275 TFLOPS pour les opérations bfloat16.
Mémoire HBM : 32 GB par puce.
Pods : 4 096 puces TPU v4 interconnectées = plus d'un pétaflop de puissance de calcul.
Consommation : optimisée, bien meilleure efficacité (FLOPS/Watt) que les GPU équivalents.

Accès aux TPU

Les TPU ne s'achètent pas. Ils sont disponibles uniquement via le cloud Google (GCP) à la location.
Google Colab : accès gratuit limité à un TPU v2.
GCP (Vertex AI) : environ 2 à 5 \$/heure pour un TPU v4.
Google Brain (recherche) : accès prioritaire pour les partenaires académiques.

III.1 — Comparatif complet : CPU vs GPU vs TPU

Voici un tableau synthétique pour choisir la bonne unité de calcul selon votre tâche.

Critère	CPU	GPU	TPU
Nb de cœurs	4 – 128	1 000 – 10 000+	Milliers de MAC units
Type de calcul	Séquentiel complexe	Parallèle massif	Parallèle tensoriel pur
Entraînement DL	✗ Très lent	✓ Excellent	🏆 Ultra-optimisé
Inférence	✓ Bon	✓ Très bon	✓ Excellent
Prétraitement données	🏆 Idéal	✓ Possible	⚠ Non recommandé
Logique de contrôle	🏆 Idéal	✓ Possible	✗ Non adapté
Mémoire utilisée	RAM (jusqu'à 2+ TB)	VRAM (8–80+ GB)	HBM intégrée (16–32 GB)
Compatibilité frameworks	Tous (PyTorch, TF...)	PyTorch, TensorFlow, JAX	TensorFlow, JAX uniquement
Accès matériel	Acheté / cloud	Acheté / cloud	Cloud uniquement (GCP)
Coût entrée de gamme	200 – 1 000 \$	350 – 2 000 \$	~2 \$/heure (GCP)

III.2 — Les goulots d'étranglement : optimiser le système entier

Matériel IA

En informatique, un goulot d'étranglement (bottleneck) est le composant le plus lent d'un système — il limite les performances de TOUS les autres. En IA, identifier et corriger ces goulots est aussi important que choisir un bon GPU.

 **Principe fondamental : votre infrastructure vaut ce que vaut son composant le plus lent.**

Goulot CPU → GPU

● Symptôme : Le GPU tourne à 30–50% d'utilisation alors qu'il est censé être saturé.

🔴 Cause : Le CPU n'arrive pas à préparer et envoyer les données au GPU assez vite. Le GPU attend les données.

✅ Solution : Augmenter le nombre de cœurs CPU. Utiliser plusieurs workers de chargement (DataLoader num_workers=8). Optimiser le prétraitement des données avec des bibliothèques comme NVIDIA DALI.

Goulot Stockage (I/O)

● Symptôme : Le GPU et le CPU sont presque inactifs. Le système attend des données depuis le disque.

🔴 Cause : Disque HDD ou SSD SATA trop lent pour alimenter les batchs d'entraînement à la vitesse requise.

✅ Solution : Passer à NVMe Gen4 (7 Go/s). Précharger les données en RAM (si dataset tient). Utiliser du stockage distribué (Ceph, HDFS) pour les très gros datasets.

Goulot PCIe (Bus de données)

● Symptôme : Transferts CPU↔GPU lents même avec un bon CPU et un bon GPU.

🔴 Cause : Bande passante PCIe insuffisante. PCIe x8 à la place de x16 divise par 2 la vitesse de transfert.

✅ Solution : Utiliser des slots PCIe 4.0 x16. Avec plusieurs GPU : vérifier que la carte mère supporte la bifurcation. Utiliser NVLink pour connecter directement les GPU entre eux (contourne le CPU).

Goulot RAM vs VRAM

● Symptôme : 'CUDA out of memory' — le modèle ne peut pas se charger. Ou swap excessif depuis le disque.

🔴 Cause : VRAM insuffisante pour le modèle + batch. RAM système insuffisante pour le prétraitement.

✅ Solution : Réduire la taille du batch. Utiliser mixed precision (float16 au lieu de float32) pour diviser par 2 l'usage VRAM. Augmenter la RAM système (min 2× la VRAM totale installée).

III.2 — PCIe : le bus de données CPU↔GPU expliqué

PCI Express (PCIe) est l'interface standardisée qui connecte le GPU (et d'autres cartes) à la carte mère et donc au CPU. Sa bande passante conditionne directement la vitesse à laquelle les données peuvent être envoyées au GPU pour l'entraînement.

Évolution des versions PCIe (bande passante par ligne × 16) :

PCIe 3.0	32 Go/s (×16)	2010 · Standard actuel dans beaucoup de workstations encore en service.
PCIe 4.0	64 Go/s (×16)	2017 · Doublement de la bande passante. Standard des cartes mères modernes.
PCIe 5.0	128 Go/s (×16)	2022 · Dernier standard. Serveurs IA modernes. Requis pour les GPU H100.

Configurations courantes et impact sur les performances :

1 GPU × ×16	Pleine bande passante. Configuration optimale pour un seul GPU. Optimal	2 GPU × ×8 chacun	Chaque GPU reçoit la moitié de la bande passante. Peut suffire pour entraînement distribué. Acceptable
4 GPU × ×4 chacun	Forte limitation du débit. Les GPU attendent les données. Inefficace sauf si NVLink disponible. Problématique	GPU ×16 + NVLink	Les GPU communiquent directement entre eux, contournant le CPU. Solution pour clusters Meilleur

III.2 — RAM, VRAM et stockage : gérer la mémoire en IA

Matériel IA

La mémoire est la ressource la plus souvent sous-estimée dans les projets IA. Comprendre la différence entre RAM, VRAM et stockage — et savoir les dimensionner — évite les erreurs les plus fréquentes.

RAM Système

16 →
512+ GB
→

Stocke les données de travail actives : dataset en cours de chargement, résultats de prétraitement, variables Python. Le CPU y accède directement.

Débits :

DDR4: 25–51 Go/s
DDR5: 51–76 Go/s

Points clés :

- Minimum recommandé : 2× la VRAM totale des GPU installés
- Si dataset < 50 GB : charger tout en RAM pour accélérer l'entraînement
- Swap (échange RAM ↔ disque) = catastrophe pour les performances
- ECC RAM (Error-Correcting Code) recommandée pour les serveurs de production

VRAM (GPU)

8 → 80+
GB
→

Mémoire dédiée du GPU. Doit contenir : paramètres du modèle + activations + gradients + batch de données. C'est LE facteur limitant pour la taille des modèles.

Débits :

GDDR6: 336 Go/s
HBM3: 3,350 Go/s

Points clés :

- float32: 4 octets/paramètre. Modèle 1B params = ~4 GB VRAM minimum
- float16 / bfloat16: 2 octets/paramètre. Divise par 2 l'usage VRAM
- Gradient checkpointing : recalcule les activations pour économiser la VRAM
- Plusieurs GPU via NVLink = cumul de VRAM (2× A100 = 160 GB effectif)

Stockage NVMe

SSD
NVMe
NVMe:

Stocke datasets, checkpoints et logs. La vitesse de lecture détermine si le GPU peut être alimenté en données assez vite. Un HDD peut diviser les performances par 70.

Débits :

HDD: 0.1 Go/s
SATA SSD: 0.55 Go/s
NVMe Gen4: 7 Go/s
NVMe Gen5: 14 Go/s

Points clés :

- HDD : inacceptable pour l'entraînement IA (trop lent)
- NVMe Gen4 (7 Go/s) : recommandé pour tous les projets IA
- RAID 0 de plusieurs NVMe : cumule les débits pour les très gros datasets
- Règle : disque doit alimenter le GPU assez vite pour maintenir > 90% GPU utilization

III.3 — Coûts matériels : CapEx, OpEx et comparaisons

Matériel IA

Comprendre les coûts d'une infrastructure IA est crucial pour prendre les bonnes décisions. Il faut distinguer les coûts d'achat (CapEx) des coûts de fonctionnement (OpEx) qui s'accumulent sur la durée.

💰 CapEx — Coûts d'investissement (achat)

GPU Gaming (RTX 4090)	~1 800 \$	Bon pour recherche/prototypage personnel
-----------------------	-----------	--

GPU Pro (A100 40GB)	~10 000 \$	Entrée de gamme datacenter IA
---------------------	------------	-------------------------------

GPU Pro (H100 80GB)	~30 000 \$	Standard pour entraîner des LLM
---------------------	------------	---------------------------------

Serveur 8× H100	> 300 000 \$	Configuration pour gros modèles
-----------------	--------------	---------------------------------

CPU Workstation (EPYC)	2 000–8 000 \$	Processeur serveur IA
------------------------	----------------	-----------------------

RAM serveur 256 GB DDR5	800–2 000 \$	Configuration standard serveur IA
-------------------------	--------------	-----------------------------------

NVMe 4 TB Gen4	300–600 \$	Stockage rapide pour datasets
----------------	------------	-------------------------------

⚡ OpEx — Coûts de fonctionnement (mensuels)

Électricité (8× GPU H100)	~3 000–5 000 \$/mois	200–400W × 8 × 24h × 30j × 0,20\$/kWh
---------------------------	----------------------	---------------------------------------

Refroidissement datacenter	30–50% du coût élec.	Indice PUE: coût total ÷ coût IT
----------------------------	----------------------	----------------------------------

Maintenance préventive	2–5% du CapEx/an	Nettoyage, remplacement pièces usées
------------------------	------------------	--------------------------------------

Connexion réseau 10 Gbps	~200–500 \$/mois	Pour transferts de données intensifs
--------------------------	------------------	--------------------------------------

Personnel IT dédié	5 000–10 000 \$/mois	Pour clusters > 10 serveurs
--------------------	----------------------	-----------------------------

Licences logicielles	Variable	CUDA (gratuit), certains outils MLOps
----------------------	----------	---------------------------------------

III.3 — Cloud computing vs Infrastructure on-premise

L'une des décisions stratégiques les plus importantes pour un projet IA : acheter son propre matériel (on-premise) ou louer de la puissance dans le cloud ? Cette décision a un impact financier et opérationnel majeur.

On-Premise (Infrastructure propre)

✓ Avantages :

- Contrôle total : données confidentielles ne quittent jamais l'entreprise
- Moins cher à long terme si utilisation intensive (> 70% en continu)
- Latence nulle entre les composants d'un même serveur
- Personnalisation maximale de l'infrastructure

⚠ Inconvénients :

- Investissement initial très élevé (CapEx > 100 000 \$ pour un setup sérieux)
- Maintenance, refroidissement, espace physique requis
- Mise à l'échelle lente : commander → livrer → installer (semaines)

Cloud Computing (AWS, GCP, Azure)

✓ Avantages :

- Pas d'investissement initial : paiement à l'usage (heure, minute)
- Scalabilité instantanée : 1 GPU → 1000 GPU en quelques minutes
- Accès aux derniers GPU (H100, TPU v4) sans achat
- Idéal pour les projets non continus (pics d'entraînement ponctuels)

⚠ Inconvénients :

- Plus coûteux à long terme si utilisation continue (> 70%)
- Données sensibles transmises à un tiers
- Dépendance au fournisseur cloud (vendor lock-in)

III.4 — Espérance de vie et obsolescence des composantes

En IA, il faut distinguer deux durées : la durée de vie physique (combien de temps le matériel fonctionne) et la durée d'utilité (combien de temps il reste compétitif face aux modèles IA modernes). La seconde est souvent bien plus courte.

Composante	Durée de vie physique	Durée d'utilité IA	Contexte d'obsolescence
CPU (Workstation)	8–12 ans	5–7 ans	Les nouvelles architectures (EPYC Genoa, Intel Sapphire Rapids) améliorent peu les perfs en IA. Le CPU tient plus longtemps.
GPU (Grand public)	5–8 ans	3–4 ans	Les nouvelles générations (RTX 3000 → 4000 → 5000) doublent parfois les performances. Un GPU de 3 ans peut ne plus supporter les nouveaux LLM.
GPU (Data Center)	4–6 ans	2–4 ans	Utilisation intensive (24h/24) = chaleur constante = usure rapide. Remplacement souvent motivé par les nouvelles architectures NVLink/NVSwitch.
TPU (Cloud)	—	Géré par Google	Pas de gestion physique. Google met à jour ses TPU (v3 → v4 → v5) et migre les clients automatiquement.
RAM DDR5	10–15 ans	8–10 ans	La RAM vieillit bien. Le changement se fait par nécessité (capacité insuffisante) plus que par obsolescence.
SSD NVMe	5–10 ans	5–7 ans	Limité par TBW (Terabytes Written). Un SSD entreprise 4TB supporte 7 PB d'écriture. En usage intensif : surveiller la santé via SMART.

Stratégie recommandée : amortir le matériel sur 3 ans en comptabilité IA, même si physiquement fonctionnel. Budgéter un remplacement GPU tous les 3 ans dans les projets de production.

III.4 — Stratégies pour optimiser les coûts d'infrastructure IA

Matériel IA

Il existe plusieurs stratégies complémentaires pour réduire les coûts sans sacrifier les performances. Une bonne infrastructure IA combine souvent plusieurs de ces approches.

01 Mixed Precision Training

Utiliser float16 ou bfloat16 au lieu de float32 pour les calculs d'entraînement. Divise par 2 l'usage VRAM et accélère les calculs sur les Tensor Cores NVIDIA.

Gain : -50% VRAM, +2x vitesse, qualité quasi identique

```
#PyTorch
from torch.cuda.amp import autocast
with autocast():
```

02 Quantisation des modèles

Réduire la précision des poids d'un modèle (32 bits → 8 bits → 4 bits) pour l'inférence. Un modèle de 70B paramètres en float32 = 280 GB. En 4-bit = 35 GB.

Gain : -75% à -87% taille modèle, inférence sur matériel moins puissant

```
# bitsandbytes
model = AutoModel.from_pretrained(
    'llama-70b',
```

03 Gradient Checkpointing

Au lieu de stocker toutes les activations en VRAM pendant la passe avant, on les recalculé à la volée lors de la rétropropagation. Échange du temps de calcul contre de la VRAM.

Gain : -60% à -70% VRAM, au prix de ~30% de temps supplémentaire

```
# PyTorch
model.gradient_checkpointing_enable()
# Réduit automatiquement
```

04 Spot/Preemptible Instances (Cloud)

Les cloud providers vendent leurs GPU inutilisés à - 70/90% du prix normal. Ces 'instances spot' peuvent être interrompues mais sont idéales pour l'entraînement avec checkpoints réguliers.

Gain : -70% à -90% du coût cloud pour l'entraînement

```
# AWS Spot GPU Instance
# g5.xlarge: 1.006 $/h (spot)
# vs 4.096 $/h (on-demand)
```

05 Model Parallelism & Distributed Training

Répartir l'entraînement sur plusieurs GPU (Data Parallelism) ou découper le modèle sur plusieurs GPU (Model Parallelism). Évite d'acheter un seul GPU très cher.

Gain : 4x GPU de 24 GB = équivalent ~96 GB VRAM pour 4x moins cher

```
# PyTorch DDP
torch.distributed.launch
```

06 ⚡ Inférence ≠ Entraînement

Une fois le modèle entraîné, l'inférence (utilisation du modèle) nécessite 10x moins de ressources. On peut deployer sur CPU, petits GPU, ou NPU embarqués.

Gain : modèle entraîné sur A100 déployable sur RTX 3060 ou même CPU

```
# Inférence CPU avec ONNX
import onnxruntime as ort
session = ort.InferenceSession
```

IV — QUIZ — Questions de compréhension

Pour chaque question, identifiez la bonne réponse avant de passer à la slide suivante.

1 Qu'est-ce que la VRAM d'un GPU et pourquoi est-elle si importante en IA ?

- A) La vitesse de calcul du GPU en TFLOPS
- B) La mémoire embarquée dans le GPU : tout le modèle + batch doit y tenir
- C) La connexion entre le GPU et la carte mère
- D) Le nombre de cœurs CUDA disponibles

2 Quelle est la différence principale entre CI/CD classique et MLOps ?

- A) MLOps est plus rapide
- B) MLOps gère uniquement le déploiement, pas l'intégration
- C) MLOps versionne aussi les données, les modèles et les expériences
- D) Il n'y a pas de différence, c'est la même chose

3 Pourquoi un ratio de 4 à 8 cœurs CPU par GPU est-il recommandé ?

- A) Pour réduire la consommation électrique
- B) Éviter que le CPU crée un goulot et laisse le GPU en attente
- C) Pour augmenter la VRAM disponible
- D) C'est une exigence de la licence CUDA NVIDIA

4 Quelle technique permet de diviser par 2 l'usage VRAM sans changer le matériel ?

- A) Ajouter plus de RAM système
- B) Remplacer le SSD par NVMe
- C) Mixed Precision Training (float16 au lieu de float32)
- D) Augmenter le nombre de lignes PCIe

IV — QUIZ — Réponses commentées

1

✓ B) La mémoire embarquée dans le GPU : tout le modèle + batch doit y tenir

La VRAM est la mémoire interne du GPU. Elle est critique car elle doit contenir simultanément les poids du modèle, les activations de chaque couche et le batch en cours. Si le modèle ne tient pas en VRAM → erreur 'CUDA out of memory'.

2

✓ C) MLOps versionne aussi les données, les modèles et les expériences

En développement classique, on versionne uniquement le code. En IA, le résultat dépend aussi des données et des hyperparamètres. MLOps ajoute DVC (données), MLflow (expériences) et un registre de modèles.

3

✓ B) Éviter que le CPU crée un goulot et laisse le GPU en attente

Le CPU prépare les données (chargement, prétraitement, augmentation) avant de les envoyer au GPU. Si le CPU est trop lent, le GPU attend des données et tourne à 30–50% d'utilisation seulement — le GPU coûteux est gaspillé.

4

✓ C) Mixed Precision Training (float16 au lieu de float32)

Passer de float32 (4 octets/valeur) à float16 (2 octets/valeur) divise par 2 l'usage VRAM et accélère les calculs sur les Tensor Cores. La précision du modèle final est quasiment identique avec le gradient scaler.

IV — QUIZ — Questions avancées (bonus)

5

Quel outil utilise-t-on pour versionner des datasets volumineux (plusieurs Go) dans un pipeline MLOps ?

- A) Git (directement dans le dépôt)
- B) DVC (Data Version Control)
- C) GitHub LFS uniquement
- D) Kubernetes

6

Pourquoi les TPU ne peuvent-ils pas être achetés par des particuliers ou des entreprises ?

- A) Ils sont trop grands pour les data centers classiques
- B) Ils ne fonctionnent qu'avec PyTorch
- C) Google les vend uniquement en tant que service cloud (GCP)
- D) Leur consommation électrique est trop élevée

7

Qu'est-ce que le NVLink de NVIDIA et quel problème résout-il ?

- A) Un protocole réseau pour connecter des serveurs
- B) Une interface directe GPU↔GPU qui contourne le goulot PCIe pour les multi-GPU
- C) Un système de refroidissement liquide
- D) Un outil de monitoring de la VRAM

8

Pour un modèle de 7 milliards de paramètres en float32, quelle VRAM minimum est nécessaire ?

- A) ~7 GB (1 octet par paramètre)
- B) ~14 GB (2 octets par paramètre en float16)
- C) ~28 GB (4 octets par paramètre en float32)
- D) ~56 GB (en comptant les gradients × 2)

IV — QUIZ — Réponses avancées

5

✓ B) DVC (Data Version Control)

DVC est le Git des données. Il crée des fichiers '.dvc' versionnables (légers, dans Git) qui pointent vers les vraies données stockées sur S3, GCS, etc. Permet de reproduire exactement une expérience passée.

6

✓ C) Google les vend uniquement en tant que service cloud (GCP)

Les TPU sont des ASIC (circuits spécifiques) propriétaires de Google. Ils ne sont disponibles qu'en location via Google Cloud Platform. Avantage : pas de maintenance, dernière génération disponible automatiquement.

7

✓ B) Une interface directe GPU↔GPU qui contourne le goulot PCIe pour les multi-GPU

NVLink permet aux GPU de communiquer directement entre eux à très haute vitesse (600 Go/s pour NVLink 4.0), sans passer par le CPU/PCIe. Essentiel pour l'entraînement distribué de très grands modèles.

8

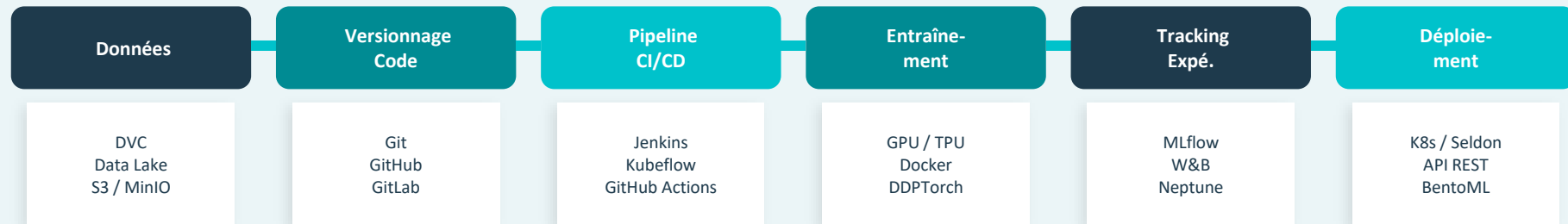
✓ C et D sont tous deux valides selon le contexte

En float32 (4 octets) : $7B \times 4 = \sim 28$ GB juste pour les poids. Mais en entraînement, il faut aussi les gradients ($\times 2$) et les activations, soit ~ 56 – 84 GB. En float16 : moitié moins. C'est pourquoi les LLM nécessitent plusieurs GPU.

V — Synthèse : l'écosystème complet d'un projet IA

Conclusion

Un projet IA performant combine une infrastructure bien choisie à chaque étape de la chaîne, de la collecte des données jusqu'à le déploiement du modèle.



Configuration matérielle recommandée selon les usages :

Apprentissage & Prototypage

CPU : 8 cœurs (Ryzen 7 / i7)
GPU : 1× RTX 4090 (24 GB VRAM)
RAM : 64 GB DDR5
Stockage : 2 TB NVMe Gen4
Budget : ~3 000 - 5 000 \$

Production IA (PME)

CPU : 32 cœurs (EPYC / Xeon)
GPU : 2-4× A100 40 GB
RAM : 256 GB ECC
Stockage : 4 TB NVMe RAID
Budget : ~40 000 - 80 000 \$

Entraînement LLM (Enterprise)

CPU : 64+ cœurs multi-sockets
GPU : 8× H100 80 GB (NVLink)
RAM : 1 TB+ ECC
Stockage : 20 TB NVMe distribué
Budget : > 300 000 \$

CONCLUSION

L'IA exige une infrastructure dédiée : calcul parallèle (GPU/TPU), données rapides (NVMe) et réseau haut débit.

La chaîne CI/CD IA (MLOps) doit versionner le code (Git), les données (DVC) et les expériences (MLflow) pour garantir la reproductibilité.

Le GPU est le cœur de l'entraînement. La VRAM est son facteur limitant. Le CPU prépare les données — si trop lent, il bride le GPU.

Les optimisations (PCIe, ratio CPU/GPU, NVMe, mixed precision) sont aussi importantes que le choix du GPU lui-même.

Coût et espérance de vie guident les décisions : 3 ans de durée d'utilité IA pour un GPU, coût cloud vs on-premise selon l'usage.

Glossaire — Termes clés à retenir

Référence

ASIC	Application-Specific Integrated Circuit. Puce électronique conçue pour une tâche unique (ex: TPU pour l'IA).	Bottleneck	Goulot d'étranglement. Le composant le plus lent d'un système qui limite les performances de tous les autres.
CUDA	Compute Unified Device Architecture. Plateforme NVIDIA permettant d'utiliser les GPU pour des calculs généraux (Deep Learning).	Data Lake	Stockage brut non structuré pour les données dans leur format original (images, textes, logs...).
DVC	Data Version Control. Outil de versionnage des données, fonctionne comme Git mais pour les datasets.	ECC RAM	Error-Correcting Code RAM. Mémoire qui détecte et corrige automatiquement les erreurs bit. Obligatoire en production.
FLOPS	Floating Point Operations Per Second. Mesure de puissance de calcul. 1 TFLOPS = 1 000 milliards d'opérations par seconde.	float16/bfloat16	Formats de nombre sur 16 bits (vs 32 bits standard). Divisent par 2 l'usage VRAM en mixed precision.
MLOps	Machine Learning Operations. Adaptation des pratiques CI/CD aux pipelines de Machine Learning.	NVLink	Interface NVIDIA permettant la communication directe GPU↔GPU à très haute bande passante, contournant le PCIe.
PCIe	PCI Express. Bus de données connectant GPU et CPU. Plus de lignes = plus de bande passante pour transferts.	TPU	Tensor Processing Unit. ASIC de Google optimisé pour les calculs tensoriels (IA). Disponible uniquement via GCP.
VRAM	Video RAM. Mémoire embarquée dans le GPU. Doit contenir modèle + batch + gradients lors de l'entraînement.		